Guía de Estudio: Common LISP y los Fundamentos del Paradigma Funcional

Esta guía articula la sintaxis fundamental de Common LISP con los conceptos nucleares del paradigma funcional.

1. Fundamentos de Common LISP: Sintaxis, Evaluación y Memoria

Common LISP utiliza notación prefija (polaca) basada en S-expresiones (*S-expressions*), donde el operador precede a sus operandos `(operador arg1 arg2 ...)`.

Evaluación y Supresión de Evaluación (*quote*)

El entorno evalúa por defecto cualquier lista considerándola una llamada a función. Si se requiere tratar la lista o un símbolo como dato literal, se debe suprimir su evaluación utilizando el operador especial `quote` o su forma abreviada `'`:

```
(+ 2 3)   ; 2 + 3

(* 2 (+ 4 5))   ; 2 * (4 + 5)

(quote (1 2 3))   ; Equivalente a '(1 2 3)
```

Salida por pantalla (print**,** format**):**

- print: Imprime el objeto evaluado en una nueva línea precedido de un salto de línea.
- format: Permite salidas estructuradas. (format t "~A~%" valor) imprime el valor seguido de salto de línea sin comillas de escape.

```
(print (+ 2 3))

(print '(+ 2 3))

(print (quote (+ 2 3)))

(format t "~A~%" (+ 2 3))

(format t "~A~%" '(+ 2 3))
```

La función ***format*** es la herramienta nativa de Lisp para estructurar y mostrar texto. Funciona mediante parámetros fijos:

- format: Invoca la función de formateo de cadenas.
- t : (Abreviatura de Terminal / True). Indica el destino de la salida. Al poner t, le ordenas a Lisp que imprima directamente en la salida estándar (pantalla/consola) en lugar de guardar el texto en una variable.

- "~A~%": Es la cadena de control (la plantilla). Los caracteres que empiezan con ~ son directivas especiales:
    - ~A (Aesthetic): Es un marcador de posición dinámico. Le dice a Lisp: "Toma el siguiente argumento disponible (en este caso, el 5) e imprímelo en un formato estéticamente legible para humanos" (sin comillas ni decoraciones).
    - ~%: Es la directiva para un salto de línea (equivalente al \n de otros lenguajes).

Ámbito de Variables: Global vs. Local

- Variables globales/especiales: Definidas con defvar o defparameter (habitualmente nombradas mediante earmuffs, e.g., *mi-global*).
- Ligaduras locales (let): Introducen un entorno léxico acotado. Los nombres solo existen dentro del cuerpo de la expresión:

```
;; 1. Declaración de variables globales (Especiales)
;; Usamos asteriscos (*...*) por convención ("earmuffs") para indicar que son globales.
(defparameter *base* 10)
(defparameter *altura* 5)

;; 2. Operación utilizando las variables globales
(format t "~A~%" (/ (* *base* *altura*) 2)) ; Devuelve 25.

;; A diferencia de 'let', estas variables SÍ existen y mantienen su valor fuera.
(format t "La base global sigue siendo: ~A~%" *base*)
```

```
(print (let ((base 10)

     (altura 5))

  (/ (* base altura) 2))) ; Devuelve 25. 'base' y 'altura' no existen fuera.
```

Definición de funciones (defun**)**

Declara una función estableciendo su nombre, lista de parámetros y cuerpo de evaluación.

(defun nombre-funcion (param1 param2)

cuerpo-de-la-funcion)

```
( defun saludar()
        (print "Hola mundo!")  )

(saludar)
```

```
;; Funcion que suma dos números

( defun sumar (a b)

   (+ a b) )

(print(sumar 2 3))
```

```
;; Saludar a <nombre de usuario>

( defun saludarUsuario(usuario)

   (print (concatenate 'string "hola " usuario ", como estas?")) )


(saludarUsuario "Antonio")
```

Parámetros opcionales (&optional): Permite declarar argumentos con valor por defecto, ideal para acumuladores o contadores de posición sin obligar al usuario a pasarlos en la llamada inicial:

(defun funcion (lista &optional (contador 1)) ...)

Bifurcaciones condicionales (if **vs.** cond**)**

- **if:** Recibe exactamente una condición, una cláusula verdadera y una falsa (if test then else).
- **cond:** Permite evaluar múltiples casos ordenados; evalúa ramas (predicado accion) secuencialmente. La rama (t accion) actúa como caso por defecto o *else*.

```
;;; --- EJEMPLO CON 'IF' ---

(defparameter *edad* 20)

(if (>= *edad* 18)

   (format t "Eres mayor de edad.~%")    ; Se ejecuta si es verdadero

   (format t "Eres menor de edad.~%"))   ; Se ejecuta si es falso (nil)
```

```
;;; --- EJEMPLO CON 'COND' ---

(defparameter *nota* 15)

(cond

  ((>= *nota* 19) (format t "Excelente~%"))

  ((>= *nota* 14) (format t "Aprobado~%"))

  ((>= *nota* 11) (format t "Suficiente~%"))

  (t                    (format t "Reprobado~%"))) ; El 't' al final actúa como el 'else' por defecto
```

Formas Lambda (lambda**):**

Funciones anónimas que se definen sin ligarse a un nombre mediante (lambda (params) cuerpo).

```
;;; --- CALCULAR EL ÁREA DE UN RECTÁNGULO INMEDIATAMENTE ---

(print ((lambda (base altura) (* base altura)) 10 5))

;; Salida en consola: 50
```

2. Anatomía de las Listas:

En LISP, toda lista se construye mediante pares enlazados denominados celdas cons.

- cons: Agrega un elemento al inicio de una lista existente (cons elemento lista).
- list: Agrupa los argumentos dados en una nueva lista (list 1 2 3) → '(1 2 3).

```
(print (cons 5 '(10 20)) )    ; (5 10 20) -> construye una nueva celda

(print (list 1 2 3) )  ; '(1 2 3)
```

Cada celda consta de dos punteros:

- **car** (*Contents of Address part of Register* / Cabeza): Primer elemento.
- **cdr** (*Contents of Decrement part of Register* / Cola): Resto de la lista.

Lista '(10 20 30):

[car: 10 | cdr: ---> [car: 20 | cdr: ---> [car: 30 | cdr: NIL]]]

```
(print (car '(10 20 30)) )    ; 10  (sinónimo: first)
(print (cdr '(10 20 30)) )    ; (20 30) (sinónimo: rest)
```

Predicados de lista y paridad:

- null: Devuelve T si el argumento es la lista vacía () o NIL, y NIL en caso contrario. Esencial para comprobar el caso base de una recursión.
- evenp / oddp: Devuelven T si un entero es par o impar, respectivamente.

3. Los 5 Pilares del Paradigma Funcional en LISP

Pilar 1: Transparencia Referencial y Funciones Puras

Una función bajo el paradigma funcional carece de efectos laterales y satisface la propiedad de que, ante los mismos argumentos, siempre genera la misma salida. Esto permite sustituir cualquier llamada por su valor sin alterar el comportamiento global.

```
;; Impuro: depende y muta un estado externo
(defvar *acumulador* 0)
(defun acumular-impuro (x)
  (setf *acumulador* (+ *acumulador* x)))


;; Puro: determinista y sin efectos secundarios
(defun sumar-puro (a b)
  (+ a b))

(print (sumar-puro 2 3) )
```

Pilar 2: Ausencia de Estado Mutable y Recursividad

En el modelo puramente funcional no se emplean celdas de memoria reasignables *(x = x + 1)* ni bucles de control de flujo imperativo (for, while). El control de flujo se modela mediante recursividad pura, avanzando sobre la lista con cdr y reduciendo el problema hasta un caso base.

Patrón con Acumulador (Recursión de Cola / Helper): Al devolver pares o valores dependientes de un índice sin mutar contadores, se emplea un parámetro auxiliar acumulado:

```
;; Retorna el primer valor par y el índice (1-based) donde se encontró
(defun primer-par-pos (lista &optional (pos 1))
  (cond
    ((null lista) nil)
    ((evenp (car lista)) (list (car lista) pos))
    (t (primer-par-pos (cdr lista) (+ pos 1)))))

;; (primer-par-pos '(1 3 8 7)) -> (8 3)
(print (primer-par-pos '(1 3 8 7)))
```

Explicación:

- *(cond ...):* Inicia la estructura de control de condiciones.
- *((null lista) nil):* Caso base 1. Si recorremos toda la lista y se queda vacía (nil) sin haber encontrado ningún número par, la función se rinde y devuelve nil.
- *((evenp (car lista)) (list (car lista) pos)):* Caso de éxito. Evaluamos el primer elemento actual (car lista). Si es par, la función se detiene inmediatamente y construye una lista de dos elementos: el número par encontrado y la posición actual (pos).
- *(t (primer-par-pos (cdr lista) (+ pos 1))):* Caso recursivo. Si el número es impar, avanzamos al resto de la lista (cdr lista) y le sumamos 1 al contador (+ pos 1) para la siguiente llamada.

Pilar 3: Funciones como Ciudadanos de Primera Clase y Orden Superior

Las funciones pueden asignarse a variables, pasarse como argumentos y devolverse como valores de retorno. Common LISP diferencia el espacio de nombres de funciones y variables, por lo que para referenciar un objeto función se utiliza la forma especial #' (lectura de la celda de función):

- funcall: Invoca una función recibiendo sus argumentos de manera explícita.
- apply: Invoca una función pasando sus argumentos contenidos en una lista.

```
(defun aplicar-operacion (fn a b)
  (funcall fn a b))

(print (aplicar-operacion #'+ 10 5) )   ; 15
(print (aplicar-operacion #'* 10 5) )   ; 50
(print (apply #'+ '(1 2 3 4 5))         )   ; 15
```

Mapeo y Transformación (mapcar, reduce y lambdas):

- **mapcar**: Toma una función y una lista. Aplica esa función a cada elemento uno por uno y genera una **nueva lista** con los resultados.
- **reduce**: Toma una función binaria (que acepta dos argumentos) y una lista. Combina todos los elementos de la lista aplicando esa función de manera acumulativa hasta reducir la lista a un **único valor**.

```
;; mapcar aplica una función a cada elemento (equivalente a map)
(print (mapcar (lambda (x) (* x x)) '(1 2 3 4)) ) ; '(1 4 9 16)

;; reduce combina pares sucesivos usando un operador binario
(print (reduce #'+ '(1 4 9 16)) )
```

Pilar 4: Composición Funcional

Los programas complejos se diseñan encadenando transformaciones atómicas: la salida de f_1 alimenta directamente a f_2 sin almacenar estructuras temporales en variables mutables intermedias.

```
;; Elevar al cuadrado los números y sumar el resultado (composición directa)
(defun suma-cuadrados (lista)
  (reduce #'+ (mapcar (lambda (x) (* x x)) lista)))

;; (suma-cuadrados '(2 3 4)) -> 4 + 9 + 16 = 29
(print (suma-cuadrados '(2 3 4)) )
```

Pilar 5: Polimorfismo y Abstracción

LISP posee tipado dinámico y funciones paramétricamente polimórficas respecto a estructuras. Operaciones estructurales como car, cdr o funciones recursivas que no ejecutan aritmética sobre los elementos pueden operar sobre listas de números, símbolos, cadenas u otras sublistas sin restricción de tipo.

```
;; Definición: extrae los primeros N elementos de cualquier lista (sin importar el tipo de datos)
(defun tomar-n (n lista)
  (cond
    ((or (<= n 0) (null lista)) nil)                    ; Caso base: n agotado o lista vacía
    (t (cons (car lista) (tomar-n (- n 1) (cdr lista))))))); Construye lista con cabeza y recursión

;; Ejecución e impresión en pantalla con diferentes tipos de datos
(print "=== Pilar 5: Polimorfismo Estructural ===")
(print (tomar-n 2 '(10 20 30 40)))                    ; Lista de enteros -> Salida: (10 20)
(print (tomar-n 2 '("alfa" "beta" "gamma")))          ; Lista de cadenas -> Salida: ("alfa" "beta")
(print (tomar-n 2 '((1 2) (3 4) (5 6))))              ; Lista de sublistas/tuplas -> Salida: ((1 2) (3 4))
```

4. Tabla de Equivalencias: Imperativo vs. Declarativo/Funcional

Concepto	**Enfoque Imperativo (C / Java)**	**Enfoque Funcional (Common LISP)**
Flujo de Ejecución	Sentencias secuenciales for, while	Evaluación de expresiones recursivas
Variables	Celdas de memoria mutables (i++)	Ligaduras léxicas inmutables (let, argumentos)
Funciones	Subrutinas con posibles efectos colaterales	Funciones puras (caja negra matemática)
Iteración sobre colecciones	Indexación por arreglos array[i]	Procesamiento de cabezas y colas (car / cdr)
Transformación en lote	Bucles con acumuladores manuales	mapcar y reduce sobre listas